

Evaluating AI-Assisted Game Code

A versioned study of where AI-generated browser-game code remains reliable—and what happens when runnable output conflicts with the truth of the input.

RESEARCHER

Shuhan Zhang · Y3931662

MODULE

AI in Creative Practice

STUDY

Six builds + map stress test

FOCUS

Reliability · constraints · validation

PIXEL ADVENTURE

Press ENTER to Start

v6 - With Audio

CAPTURED DIRECTLY FROM THE LIVE HTML

What breaks when the model must carry logic forward?

The study moves beyond “can AI write code?” and tests whether generated logic stays coherent as one small game accumulates state, collision rules and edge cases.

RQ1

To what extent can generative AI maintain logical consistency across multiple iterations of game-code development?

RQ2

In which situations does AI-generated game code hallucinate or break input constraints?

RQ3

How can AI-assisted workflows be used safely and effectively in small-scale game development?

STUDY PROTOCOL



One evolving game

The same single-file browser prototype grows from movement to combat, UI and scenes.



One model

Gemini 3 Pro Thinking, with thinking mode enabled.



Traceable evidence

Prompt, output file, screen recording and written log for each iteration.

Microsoft Edge test environment

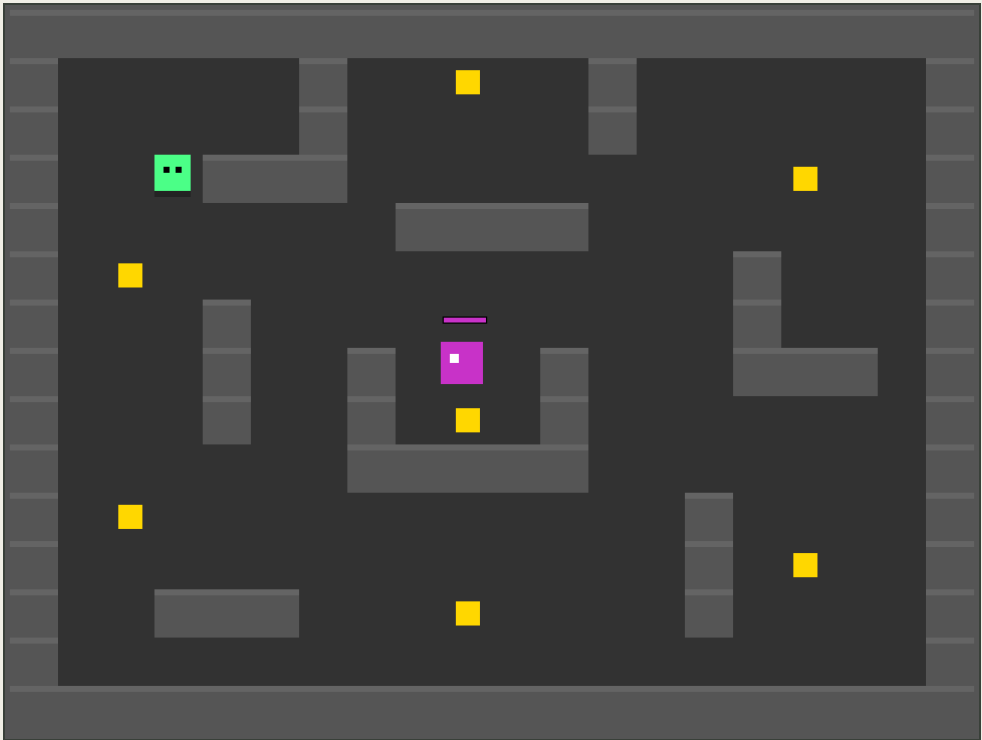
From hand-written baseline to AI-assisted systems

Implementation became dramatically faster, but each added dependency increased the need to verify what the model assumed.

BUILD / TIME	ADDED	OBSERVED BEHAVIOUR	REVIEW
v1 27 MIN · MANUAL	Baseline	Character movement and basic attack actions, written by the researcher as the comparison point.	● NO REPAIR
v2 1 MIN · AI	Replication	Same functions, with clearer naming, comments and structure than the manual baseline.	● NO REPAIR
v3 2 MIN · AI	Tilemap + collision	Collision worked, but the new spawn logic placed the player inside a wall.	● MANUAL REPAIR
v4 2 MIN · AI	Enemies + combat	HP, invulnerability timing, random movement and collision-based hit detection.	● NO REPAIR
v5 1 MIN · AI	Timing + respawn	Complete death/respawn cycle and UI state across the combat loop.	● NO REPAIR
v6 SCAFFOLD	Scenes + music	A playable extension demonstrating scope, kept outside the reliability analysis.	● DEMO ONLY

A working demo, not a reliability claim

v6 extends the prototype with a title scene and music system. It shows the reach of AI assistance while remaining explicitly outside the v1–v5 analysis.



V6 GAMEPLAY / CAPTURED FROM THE RUNNING HTML, NOT A MOCKUP

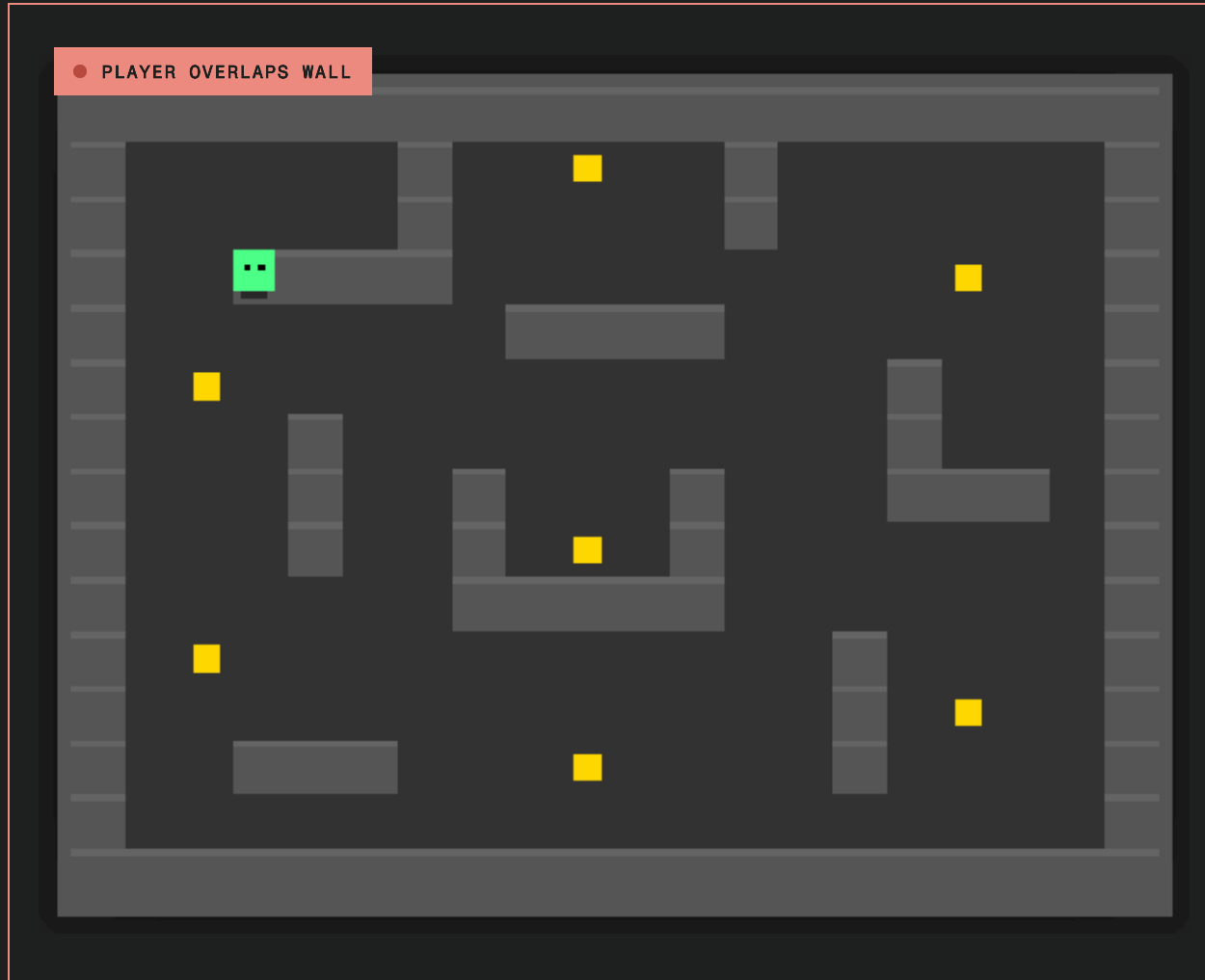
Playable does not mean fully tested.

I built the scene and audio branch after the five-version study. It demonstrates what could be assembled with AI assistance, but it was not run through the same reliability tests.

v6_scene_music_system.html		Source observations
01	Enter starts the game	<code>SceneManager.changeScene(GameScene)</code> runs only while the title scene is active.
02	Scenes own their music	<code>TitleScene.enter</code> and <code>GameScene.enter</code> request different BGM tracks.
03	Audio waits for interaction	<code>SoundManager.init()</code> runs after the user's first key press to satisfy browser playback rules.
Playable extension / outside the analysed sample		

The collision worked. The assumption did not.

The first serious failure appeared when the map system was added: the generated player spawn overlapped a solid tile, leaving the character unable to move.



WHAT SUCCEEDED

Collision logic

Solid walls correctly blocked movement—the system behaved exactly as coded.

WHAT FAILED

Spawn feasibility

The model reused or introduced spawn logic without checking whether the chosen coordinate was walkable in the new map.

Prompt asked for a 16px tile grid, solid-wall collision, four-direction movement and collectible coins—while preserving existing DOM names and returning a fully executable single-file prototype.

The all-wall map exposed a different failure

Five solvable maps test spatial reading. A sixth, made entirely of walls, tests whether the model reports an impossible condition or rewrites the input.

MAP	CONDITION	EXPECTED	MAP CHANGED
map01	Open room	Yes	No
map02	Narrow corridor	Yes	No
map03	Multiple rooms	Yes	No
map04	Dead-end maze	Yes	No
map05	Random noise	Yes	No
map06_v1	All walls · constraint implicit	No	Yes
map06_v2	All walls · mutation forbidden	Yes	No

5 / 5

Solvable maps were read correctly without modifying the tile data.

DIAGNOSTIC SHIFT

Spatial recognition was not the core problem.

The divergence appears only when no valid answer exists. The critical variable is how failure is handled—not whether walls can be recognised.

One explicit boundary changed the failure behaviour

The underlying map stayed identical. Only the instruction changed: mutation was prohibited, and honest failure was defined as an acceptable output.

SAME INPUT IN BOTH RUNS	map06 · all walls	No walkable tile; no valid spawn coordinate exists.
MAP06_V1 / CONSTRAINT IMPLICIT	Made the map solvable The prompt left the no-solution path undefined. The model changed tiles until it could return a runnable game.	MAP06_V2 / CONSTRAINT EXPLICIT Reported no spawn The revised prompt prohibited tile changes and allowed a failure response. The model left the map intact.
INPUT MUTATED		INPUT PRESERVED
CLAUSE ADDED TO THE SECOND PROMPT	"Do NOT modify or mutate the provided tilemap array... If the map contains no walkable tiles, the program must NOT carve or change tiles... set spawn = null."	

What improved, and what v3 exposed

The strongest benefit appeared in well-defined implementation work; the decisive failure emerged at the boundary between new logic and existing state.

FINDING 01

Iteration accelerates implementation

AI-generated versions added features in one to two minutes and often improved naming, comments and function separation over the manual baseline.

1–2m AI build time after the 27-minute manual baseline

FINDING 02

Spatial recognition was never the problem

Every solvable map was interpreted correctly. The v3 bug is better explained as unverified spawn logic carried into a changed environment.

Wall detection
worked

≠

Spawn feasibility
unchecked

On map06, the model **changed the problem**

The unsolvable map exposed the risk most clearly: absent a stated boundary, the model altered the problem rather than reporting failure.

FINDING 03



This behaviour can be characterised as executability-oriented generation, where the model prioritises producing runnable outputs even at the cost of violating input constraints.

ANALYSIS · SECTION 5.3

FINDING 04

Constraint awareness has to be stated

The model respected an immutable-input boundary only after the prompt made it explicit and defined acceptable failure behaviour.

Implicit boundary
data changed



Explicit boundary
failure reported

What this study changed in **my workflow**

The v3 and map06 failures made the review step more specific: check inherited assumptions, impossible inputs and unrequested changes to source data.

WORKING NOTES AFTER THE EXPERIMENT

1

Protect the input

State that the tilemap and existing interfaces cannot be silently rewritten.

2

Allow “no solution”

Specify `spawn = null` or a visible error for an all-wall map.

3

Keep the trail

Connect each prompt with its output, recording and written observation.

4

Retest after each feature

The v3 bug appeared where new collision rules met an inherited spawn assumption.

MY POSITION

The model writes candidate logic. I decide whether it holds.

The more systems the prototype gained, the more time moved from writing code to checking its boundaries.

STUDY LIMITATIONS

- Single-model study; no cross-model comparison.
- Small pixel-game environment, not a production codebase.
- Human judgement shaped repair and correctness decisions.
- Next: larger environments and automated validation tooling.

The developer did not disappear. **The role moved.**

From writing every implementation to **validating what the model decided to change.**

Clear prompts produced playable features and readable code. Misconfigured or impossible inputs exposed the deeper risk: the model may manufacture success unless invariants and acceptable failure states are explicit. With versioned evidence and human boundary checks, AI remains a useful small-scale development collaborator.

Individual project by Shuhan Zhang. Gemini 3 Pro Thinking generated v2–v6 and the extended-experiment responses from author-written prompts. All outputs were tested, screen-recorded, logged and reviewed by the author.



PLAY

Open the live build

Experience the title scene, music and playable v6 prototype.

shuhan0001.github.io/portfolio/v6_scene_music_system.html



INSPECT

View the repository

Browse the versioned code and project evidence.

github.com/Shuhan0001/Evaluating-AI-Assisted-Game-Code